DESIGN AND FPGA IMPLEMENTATION OF NOISE FILTERING AND OBJECT DETECTION TECHNIQUES FOR RADAR POINT CLOUD PROCESSING

PROJECT REPORT

Submitted in fulfillment of the requirements for the award of the Degree of ***Bachelor of Technology*** *in* ***Electronics & Communication Engineering*** *of* ***APJ Abdul Kalam Technological University***

By

ARAVIND K P (VIII SEM B.TECH, REG. NO. MDL22EC030)

C N MUHAMMED ANZIL (VIII SEM B.TECH, REG. NO. MDL22EC040)

P S RAZEEN (VIII SEM B.TECH, REG. NO. MDL22EC098)

RITHUN B REGISH(VIII SEM B.TECH, REG. NO. MDL22EC101)

APRIL 2026

DEPARTMENT OF ELECTRONICS ENGINEERING
MODEL ENGINEERING COLLEGE, THRIKKAKARA
ERNAKULAM

MODEL ENGINEERING COLLEGE, THRIKKAKARA

DEPARTMENT OF ELECTRONICS ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **" DESIGN AND FPGA IMPLEMENTATION OF NOISE FILTERING AND OBJECT DETECTION TECHNIQUES FOR RADAR POINT CLOUD PROCESSING "** *is a bonafide record of the project work done by* **ARAVIND K P** *(Reg. No. MDL22EC030),* **C N MUHAMMED ANZIL** *(Reg. No. MDL22EC040),* **P S RAZEEN** *(Reg. No. MDL22EC098),* **RITHUN B REGISH** *(Reg. No. MDL22EC101) towards the partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Electronics & Communication Engineering of APJ Abdul Kalam Technological University during the year 2026.*

PROJECT GUIDE

DR. JOBYMOL JACOB
PROFESSOR
DEPT. OF ELECTRONICS ENGG

PROJECT CO-ORDINATOR

MR. JAYADAS C K
ASSOCIATE PROFESSOR
DEPT. OF ELECTRONICS ENGG

HEAD OF DEPARTMENT

DR. JOBYMOL JACOB
DEPT. OF ELECTRONICS ENGG

ACKNOWLEDGEMENT

*First of all, we would like to thank the **Lord Almighty** who helped us to finish this project on time.*

*We extend our heartfelt thanks to **Dr. Jayachandran E. S.**, Principal, Model Engineering College, Thrikkakara, for providing the necessary facilities and academic environment for the completion of this project.*

*We express our sincere gratitude and heartfelt thanks to **Dr. Jobymol Jacob**, Head of the Department and Project Guide, Department of Electronics and Communication Engineering, for her continuous guidance, constructive feedback, encouragement, and support throughout the project, which were instrumental in the successful completion of this work.*

*We express our sincere gratitude to **Mr. Rashid M. E.**, Assistant Professor and Co-Chief Investigator of the Chip to Startup (C2S) project, for his valuable project guidance, technical support, and continuous assistance throughout the project period.*

*We sincerely acknowledge the **Chip to Startup (C2S)** programme for providing the opportunity and platform under which this project was carried out.*

*We express our sincere gratitude and heartfelt thanks to **Mr. Jayadas C. K.**, Associate Professor, and **Ms. Sheeba P. S**, Associate Professor, Department of Electronics and Communication Engineering, for their invaluable guidance, technical insights, continuous support, and constant motivation throughout the project.*

We also thank all the teaching and non-teaching staff of the department for their cooperation.

Finally, we thank our parents and friends for their constant moral support.

ARAVIND K P (MDL22EC030)

C N MUHAMMED ANZIL (MDL22EC040)

P S RAZEEN (MDL22EC098)

RITHUN B REGISH (MDL22EC101)

ABSTRACT

A real-time radar-only perception pipeline for autonomous systems is designed and implemented to address challenges such as noise, sparsity, and unreliable detections in automotive radar data. Radar sensors provide reliable operation under adverse environmental conditions; however, raw radar point clouds require advanced processing to extract meaningful object-level information. The proposed approach incorporates a complete processing pipeline using the nuScenes-mini dataset, including multi-sweep accumulation, noise filtering, density-based clustering (DBSCAN), object tracking, and threat evaluation to generate a structured Bird's Eye View (BEV) representation of the environment. To achieve real-time performance, the processing pipeline is implemented on FPGA using both frame-based and stream-based architectures, where the stream-based design, developed using the AXI4-Stream protocol and pipelined modules, enables continuous data processing with reduced latency and improved throughput. Simulation results and hardware analysis demonstrate efficient resource utilization, low power consumption, and successful timing closure, with the streaming architecture significantly enhancing real-time capability compared to conventional approaches, thus establishing the feasibility of a complete radar-only perception system and highlighting the effectiveness of FPGA-based acceleration for real-time autonomous applications.

Contents

List of Figures

List of Tables

List of Abbreviations

- ADAS – Advanced Driver Assistance Systems
- AXI – Advanced eXtensible Interface
- AXI4-Stream – Advanced eXtensible Interface Streaming Protocol
- BEV – Bird's Eye View
- BRAM – Block Random Access Memory
- DBSCAN – Density-Based Spatial Clustering of Applications with Noise
- DMA – Direct Memory Access
- DSP – Digital Signal Processing
- EKF – Extended Kalman Filter
- FPGA – Field Programmable Gate Array
- FSM – Finite State Machine
- HIL – Hardware-in-the-Loop
- IP – Intellectual Property (Core)
- LRR – Long Range Radar
- LUT – Look-Up Table
- PCD – Point Cloud Data
- PL – Programmable Logic
- PS – Processing System
- PYNQ – Python Productivity for Zynq
- Q8.8 – Fixed-Point Format (8 integer bits, 8 fractional bits)
- RADAR – Radio Detection and Ranging
- RTL – Register Transfer Level
- TLAST – AXI Signal Indicating End of Frame
- TREADY – AXI Signal Indicating Ready to Receive Data
- TVALID – AXI Signal Indicating Valid Data
- TTC – Time To Collision

Chapter 1

INTRODUCTION

Advanced Driver Assistance Systems (ADAS) have emerged as a critical component in modern vehicles, enhancing safety, comfort, and driving efficiency through intelligent perception and decision-making capabilities. These systems rely on a combination of sensing technologies such as cameras, LiDAR, and radar to perceive the surrounding environment and assist drivers in tasks such as collision avoidance, lane keeping, adaptive cruise control, and emergency braking. Among these, radar-based sensing has gained significant importance due to its robustness in challenging environmental conditions and its ability to provide reliable distance and velocity information. As the automotive industry moves toward higher levels of autonomy, the demand for accurate, real-time, and reliable perception systems becomes increasingly essential, making radar-based object detection and noise filtering techniques highly relevant for next-generation ADAS applications.

Reliable perception is a fundamental requirement for autonomous vehicles and ADAS, where radar plays a crucial role due to its ability to operate effectively under adverse environmental conditions such as fog, rain, dust, and low-light scenarios. Unlike cameras and LiDAR, automotive radar provides direct range and velocity information using electromagnetic wave reflections, making it well suited for safety-critical applications.

However, raw automotive radar data is inherently sparse, noisy, and affected by multipath reflections. Radar point clouds often contain isolated detections, static clutter, and ghost reflections that do not correspond to real physical objects, making direct object detection and tracking challenging. Therefore, efficient preprocessing, clustering, and tracking techniques are required to transform raw radar returns into meaningful object-level information.

The work focuses on the development of a radar-only perception pipeline that processes raw radar point cloud data into structured object representations, including position, velocity, and threat level, while leveraging FPGA-based hardware acceleration to achieve real-time per-

formance.

1.1 Motivation

The motivation for this work arises from the need for a dependable perception system that does not rely solely on vision-based sensors, which are highly susceptible to environmental degradation. Radar-based perception offers a robust alternative and can serve as a reliable fail-safe layer for vehicle safety in adverse conditions.

It is essential to explore the capability of radar data in independently detecting and analyzing surrounding objects using appropriate signal processing and clustering techniques. Furthermore, achieving real-time performance is critical for practical deployment in autonomous systems, motivating the transition from software-based implementation to FPGA-based hardware acceleration.

1.2 Importance of the Project

The importance of this work lies in demonstrating a complete radar-only perception workflow spanning both software and hardware domains. The system incorporates advanced radar data processing techniques such as multi-sweep accumulation and spatial filtering to address the inherent sparsity and noise present in radar data, thereby improving detection reliability.

An efficient clustering mechanism is implemented using density-based approaches, including DBSCAN in software and grid-based clustering in hardware, enabling effective grouping of radar points into meaningful object-level representations. The processing pipeline is further accelerated through FPGA-based implementation using Verilog, enabling parallel execution, deterministic performance, and reduced latency.

In addition, a streaming architecture based on AXI4-Stream is developed to support continuous data processing with improved throughput, making the system suitable for real-time applications. The complete pipeline is validated end-to-end, from Python-based data generation to FPGA execution and output retrieval on the PYNQ-Z2 platform, demonstrating the

feasibility of a hardware-accelerated radar-only perception system.

1.3 Objectives

The primary objectives of this project are:

1. Develop a radar-only perception pipeline using multi-sweep radar point cloud data from the nuScenes-mini dataset.
2. Enhance radar data quality through noise suppression, filtering, and accumulation techniques to improve detection reliability.
3. Generate object-level representations from radar point clouds through clustering, along with motion estimation and tracking.
4. Design and implement the perception pipeline on FPGA using Verilog HDL, ensuring efficient parallel processing.
5. Evaluate frame-based and stream-based architectures and enable efficient software-hardware communication using AXI DMA on the PYNQ-Z2 platform.

1.4 Scope

The scope of the project includes both software and hardware implementation of a radar-only perception pipeline. In the software phase, radar point cloud data from the nuScenes-mini dataset is processed using Python to perform noise filtering, clustering, tracking, and evaluation, with a primary focus on algorithm development and validation. The validated pipeline is subsequently mapped to hardware and implemented on FPGA using Verilog HDL, exploring both frame-based and stream-based architectures with an emphasis on AXI4-Stream based real-time processing.

The system is deployed on the PYNQ-Z2 platform to enable end-to-end execution, utilizing AXI DMA for efficient communication between the Processing System and Programmable Logic. The current implementation demonstrates clustering and centroid-based object detection on hardware, while advanced functionalities such as full object tracking and multi-sensor fusion are beyond the present scope and are identified as potential directions for future work.

1.5 Applications

The developed radar-only perception system is applicable in several safety-critical and autonomous scenarios, including:

- **All-Weather ADAS Operation:** Reliable object detection in fog, rain, and low-visibility conditions.
- **Collision Avoidance Systems:** Early detection of approaching objects using radar-based distance and motion estimation.
- **Surround Monitoring and Blind Spot Detection:** Near 360-degree perception using multiple radar sensors.
- **Autonomous Robotics:** Deployment in industrial or harsh environments where optical sensors are unreliable.

1.6 Organisation of the Report

This report is structured to present the development, implementation, and evaluation of the radar-only perception system in a systematic manner.

Chapter 2 presents a comprehensive literature review of existing research in radar signal processing, point cloud filtering, clustering techniques, and hardware acceleration approaches, providing the foundation for the proposed system.

Chapter 3 describes the design of the radar-only perception pipeline, including the problem formulation, proposed solution, system architecture, and detailed methodology for processing radar point cloud data.

Chapter 4 focuses on the software implementation of the proposed pipeline, detailing parameter configuration, clustering logic, tracking mechanisms, and visualization using Bird's Eye View (BEV).

Chapter 5 presents the FPGA-based hardware implementation of the system, including both frame-based and stream-based architectures, along with detailed module design and optimization techniques for real-time processing.

Chapter 6 discusses the testing and results, including software validation, FPGA simulation, real hardware execution on the PYNQ-Z2 platform, and performance analysis.

Finally, Chapter 7 concludes the report by summarizing the contributions of the project, discussing its limitations, and outlining future scope for further improvements.

This chapter has introduced the fundamental background, motivation, objectives, and scope of the radar-only perception system. It established the significance of radar as a reliable sensing modality and outlined the overall structure of the project, spanning both software and hardware domains. The following chapter will review existing research and methodologies related to radar data processing, clustering, and hardware acceleration, providing the necessary context for the design choices made in this work.

Chapter 2

LITERATURE REVIEW

This chapter builds upon the foundational concepts introduced in the previous chapter, where the significance of radar-based perception and the challenges associated with raw radar data were discussed. To address these challenges, it is essential to examine existing research and methodologies in radar signal processing, point cloud generation, clustering, tracking, and real-time system implementation. This chapter presents a comprehensive review of relevant literature, highlighting recent advancements and identifying key research gaps that motivate the development of a radar-only perception pipeline with hardware acceleration.

2.1 Radar Signal Processing in Autonomous Driving

Radar sensors play a critical role in autonomous driving due to their robustness under adverse environmental conditions such as fog, rain, and low-light scenarios. Automotive radar systems provide direct range and velocity measurements using electromagnetic wave reflections, making them suitable for safety-critical applications [1].

Recent advancements have focused on improving the usability of sparse radar data through high-resolution point cloud generation techniques. Methods such as RadCloud [2] demonstrate that low-cost radar sensors can produce dense and meaningful point clouds using advanced signal processing. Cooperative radar perception techniques [3] further enhance environmental understanding by fusing data from multiple radar sensors.

Emerging approaches such as diffusion-based super-resolution [4] aim to improve radar point cloud quality, while radar-only odometry methods [5] demonstrate the feasibility of using radar as a standalone sensing modality.

Despite these advancements, challenges such as noise, sparsity, and ghost detections caused by multipath reflections remain significant. Classical tracking and estimation tech-

niques [6, 7] emphasize the importance of probabilistic filtering and temporal data association for improving detection stability.

2.2 Radar Point Cloud Processing and Filtering

Efficient preprocessing of radar data is essential for reliable object detection. Radar point clouds are inherently sparse and noisy, requiring filtering techniques to isolate meaningful detections. Spatial filtering and Doppler-based thresholding are commonly used to remove static clutter and prioritize moving objects.

Adaptive filtering methods based on statistical signal processing [8] are widely used to suppress noise while preserving important signal characteristics. Multi-sweep accumulation techniques improve point cloud density by combining information across multiple frames.

Recent research has also explored radar-specific object detection techniques [9], while learning-based feature extraction approaches [10] further improve radar data representation, albeit with higher computational cost.

2.3 Clustering Techniques for Radar Data

Clustering plays a key role in transforming radar point clouds into object-level representations. Density-Based Spatial Clustering of Applications with Noise (DBSCAN) [11] is widely adopted for radar applications due to its ability to handle noise and varying point densities.

Recent studies in radar perception [12] highlight the effectiveness of clustering-based approaches. Hybrid clustering and tracking techniques [13] improve temporal consistency and object detection reliability in dynamic environments.

2.4 Real-Time Processing and Hardware Acceleration

Real-time processing is a critical requirement for autonomous driving applications. While software-based implementations provide flexibility, they often fail to meet strict latency constraints. Hardware acceleration using FPGA platforms enables parallel processing and deter-

ministic execution.

AXI-based communication protocols such as AXI DMA and AXI4-Stream [14, 15] enable high-throughput data transfer between processing units. Stream-based architectures allow continuous data flow, reducing latency and improving throughput.

FPGA-based system design approaches [16] demonstrate significant advantages in embedded signal processing. Furthermore, accurate perception supports higher-level functions such as motion planning [17].

2.5 Dataset-Driven Prototyping and Validation

The development and validation of radar perception systems rely on standardized datasets. The nuScenes dataset [18] provides a comprehensive multi-modal benchmark, including radar data.

Datasets such as KITTI [19] are widely used for benchmarking. Software-based prototyping using Python enables rapid algorithm development before deployment on hardware platforms.

This chapter reviewed key developments in radar signal processing, filtering, clustering, and real-time implementation. The literature highlights the importance of combining efficient signal processing with hardware acceleration to achieve reliable radar perception systems.

Chapter 3

DESIGN OF RADAR-ONLY PERCEPTION PIPELINE

This chapter builds upon the insights obtained from the literature review, where various radar processing techniques, clustering methods, and system design approaches were examined. Based on these findings, a structured radar-only perception pipeline is proposed to address the challenges of noise, sparsity, and unreliable detections in raw radar data. This chapter presents the problem formulation, overall system architecture, and detailed methodology used to transform raw radar point clouds into meaningful object-level representations suitable for autonomous driving applications.

3.1 Problem Statement

Autonomous vehicles face a significant perception challenge under adverse environmental conditions such as fog, heavy rain, and low-light scenarios. While cameras and LiDAR provide high-resolution environmental information, their performance degrades severely in such conditions. Radar sensors offer a reliable alternative due to their robustness to weather and their ability to directly measure object velocity.

However, raw automotive radar data is inherently sparse and contaminated with thermal noise, static clutter, and multipath-induced ghost detections. These false detections often arise from reflections off road surfaces, guardrails, and surrounding metallic structures. Conventional radar processing pipelines typically rely on simple threshold-based filtering, which either fails to remove persistent noise or inadvertently discards sparse but critical detections from distant objects.

This lack of robust filtering, clustering, and tracking logic leads to unstable object perception, increasing the risk of false alarms such as phantom braking or missed detections. Hence, a software-defined radar-only perception framework is required to reliably distinguish noise

from physical objects.

3.2 Proposed Solution

To address the identified challenges, a modular radar-only perception pipeline is proposed. The system transforms raw radar point clouds into structured object-level representations using a multi-stage, software-defined approach. The nuScenes-mini dataset is used for development and validation under realistic urban driving scenarios.

The proposed solution emphasizes robustness to noise, explainability, and suitability for future hardware acceleration. Temporal multi-sweep accumulation is employed to overcome radar sparsity, while density-based clustering and lightweight tracking enable stable object detection without reliance on vision sensors.

The processing pipeline consists of several key stages. Initially, preprocessing is performed to filter radar data using a perception radius constraint of 60 m, ensuring that only ADAS-relevant regions are considered. To further improve data quality, temporal multi-sweep accumulation is applied by combining up to five consecutive radar frames, thereby enhancing point cloud density, improving object continuity, and suppressing transient noise.

Subsequently, density-based clustering is carried out using the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm. This method groups spatially coherent radar points into object clusters while inherently rejecting isolated noise points. It is particularly well suited for radar data due to its ability to handle varying point densities and irregular object shapes.

Following clustering, detected objects are tracked across frames using centroid-based association, with persistence logic incorporated to handle temporary occlusions. In addition, threat assessment is performed by analyzing object distance, radial velocity, and spatial extent, allowing classification into Low, Medium, or High threat levels for safety-oriented evaluation.

This software-defined framework enables the generation of a clean Bird's Eye View (BEV) representation and provides a validated algorithmic baseline for future real-time deployment and FPGA-based acceleration.

3.3 System Architecture

Figure 3.1 illustrates the overall architecture of the radar-only perception pipeline. The system processes raw radar point cloud data acquired from multiple sensors positioned around the vehicle and converts it into object-level information such as position, velocity, and threat level, with each processing stage performing a clearly defined function. The modular architecture allows preprocessing, clustering, and tracking blocks to be independently analyzed, optimized, or migrated to hardware, ensuring transparency, scalability, and suitability for real-time and hardware-oriented implementations in future phases.

3.4 Methodology

3.4.1 Radar Point Cloud Loading

Radar point cloud data is loaded from the nuScenes dataset using the nuScenes-devkit. Each radar point contains spatial coordinates along with ego-compensated Doppler velocity components. Data from five radar sensors positioned around the vehicle is used to provide near 360-degree environmental coverage.

3.4.2 Multi-Sweep Accumulation

Radar data sparsity is addressed through temporal multi-sweep accumulation. Multiple consecutive radar frames are combined to increase point density, improving object detectability, especially at longer ranges.

3.4.3 Clustering

Density-based clustering is applied to group radar points belonging to the same physical object. DBSCAN is chosen due to its ability to handle arbitrary cluster shapes and varying point densities without requiring a predefined number of clusters.

3.4.4 Small Cluster Removal

Clusters with insufficient spatial extent or very few points are removed, as they typically correspond to noise or unstable reflections. This step improves robustness by retaining only meaningful object-level detections.

3.4.5 Perception Radius Filtering

A fixed perception radius is applied to discard radar points and clusters outside the region of interest. This reduces computational complexity and ensures focus on ADAS-relevant areas around the ego vehicle.

3.4.6 Bounding Box Generation

For each valid cluster, an axis-aligned bounding box is generated. Bounding boxes represent object size, position, and spatial extent and serve as the primary object representation for tracking and evaluation.

3.4.7 Cluster Tracking

Clusters are tracked across frames using centroid-based association. Each object is assigned a unique identifier, and track persistence is maintained using a miss counter to tolerate temporary occlusions.

3.4.8 Velocity Mapping

Mean Doppler velocity is computed for each tracked object using the velocity components of the radar points within the cluster. Radial velocity relative to the ego vehicle is then derived to determine motion behavior.

3.4.9 Evaluation

The final stage evaluates detection stability, tracking consistency, velocity behavior, and qualitative visualization using a Bird's Eye View interface. The evaluation framework is radar-

only and does not rely on camera or LiDAR ground truth.

3.5 Components Used

3.5.1 Software Environment and Libraries

The system is implemented in Python using a set of standard libraries tailored for radar data processing and visualization. The nuScenes-devkit is used for radar data loading and synchronization, while Scikit-Learn provides the DBSCAN algorithm for density-based clustering. NumPy is utilized for efficient numerical computation, and Matplotlib is employed for Bird's Eye View (BEV) visualization and animation of the processed radar data.

3.5.2 Dataset Used

The nuScenes-mini dataset is used for development and validation of the radar-only perception pipeline. It is a subset of the large-scale nuScenes autonomous driving dataset and provides synchronized multi-sensor data collected in real-world urban environments. The "v1.0-mini" subset is chosen as it offers adequate scene diversity with lower computational overhead.

Radar data from five 77 GHz long-range radar sensors—Front, Front-Left, Front-Right, Back-Left, and Back-Right—is utilized to achieve near 360-degree coverage around the ego vehicle. Each radar point contains spatial coordinates and ego-compensated Doppler velocity information, enabling direct motion estimation suitable for radar-only perception.

3.5.3 Core Algorithmic Modules

The radar-only perception pipeline is composed of modular algorithmic components designed to handle radar data sparsity, noise, and object motion estimation: The radar-only perception pipeline is composed of several core algorithmic modules designed to handle radar data sparsity, noise, and object motion estimation. Density-based clustering is performed using DBSCAN, which groups spatially proximate radar points into object-level clusters while inherently filtering out noise. The algorithm parameters are empirically selected based on experiments conducted on the nuScenes-mini dataset.

A dictionary-based tracking mechanism is implemented to maintain object identity across frames. This approach uses centroid-based association and assigns unique track IDs to detected objects, while a miss counter is incorporated to handle temporary occlusions and intermittent detections.

In addition, a threat assessment module evaluates tracked objects based on distance, radial velocity derived from Doppler measurements, and bounding box size. These parameters are used to classify objects into High, Medium, or Low threat levels, enabling safety-oriented decision-making within the perception system.

$$Threat = f(\text{Distance}, \text{Radial Velocity}, \text{Bounding Box Size}) \tag{3.1}$$

Objects are classified into **High**, **Medium**, or **Low** threat levels for safety-oriented decision-making.

This chapter presented the complete design of the radar-only perception pipeline, including the problem formulation, proposed solution, system architecture, and detailed methodology. The integration of preprocessing, clustering, tracking, and evaluation stages establishes a robust framework for extracting meaningful information from radar data. Additionally, the use of modular components and software-based validation provides a strong foundation for hardware implementation. The next chapter focuses on the practical implementation of this pipeline, detailing the software design and processing logic used to realize the proposed system.

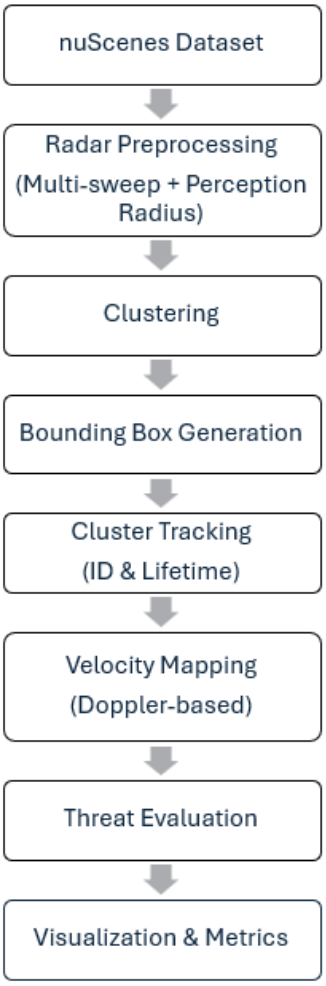

Figure 3.1: Block diagram of the radar-only perception pipeline

Chapter 4

SOFTWARE IMPLEMENTATION

This chapter detailed the implementation of the radar-only perception pipeline, including parameter configuration, radar data processing, clustering, tracking, and visualization using a Bird's Eye View representation. The software-based implementation serves as a validated reference model for the system, demonstrating the effectiveness of the proposed approach in handling radar data. Building on this foundation, the next chapter focuses on the hardware realization of the pipeline, describing the FPGA-based implementation and architectural optimizations for real-time processing.

4.1 System Parameter Configuration

The implementation of the radar perception pipeline requires precise configuration of spatial and temporal parameters to ensure reliable object detection. These parameters are defined in the Python environment and later guide the hardware implementation.

Parameter	Value	Description
PERCEPTION_RADIUS	60.0	Maximum sensing range (meters)
MAX_SWEEPS	3	Number of accumulated radar frames
SPARSE_EPS	1.0	Epsilon for initial noise filtering
OBJECT_EPS	1.5	Epsilon for object clustering
MIN_OBJECT_POINTS	5	Minimum points to form a cluster
TRACK_MATCH_DIST	2.5	Max distance for track association (m)
MAX_MISSES	3	Frames to retain a lost track

Table 4.1: Software Configuration Parameters

The configured parameters play a critical role in ensuring reliable radar perception. The perception radius limits processing to a 60-meter region around the ego vehicle, thereby reducing noise and computational overhead. Multi-sweep accumulation improves point cloud density by combining multiple radar frames, which enhances object detectability.

A dual-stage epsilon strategy is adopted, where a smaller epsilon value is used for initial noise filtering and a larger epsilon value is applied during object clustering to improve grouping accuracy. Additionally, the track match distance defines the threshold for associating clusters across frames, enabling consistent object tracking over time.

4.2 Radar Processing Logic and Clustering

The radar processing pipeline is designed to handle the sparsity and noise characteristics of automotive radar data. Raw radar points are loaded in the ego-vehicle coordinate frame and accumulated across multiple sweeps to improve density.

A perception radius constraint is applied to focus on relevant regions. Clustering is performed using the DBSCAN algorithm, which groups spatially dense points while filtering out noise.

The resulting clusters represent potential physical objects and are forwarded to the tracking module. This software implementation serves as the reference model for the FPGA-based hardware design described in later chapters.

4.3 Tracking and Threat Assessment Logic

The tracking module is implemented using a lightweight dictionary-based approach. Detected clusters are associated across frames using centroid distance matching, and each object is assigned a unique track ID.

Each track maintains parameters such as centroid position (cx, cy), bounding box dimensions, velocity, age, and miss count. Temporary occlusions are handled by retaining tracks for a limited number of frames. Threat assessment is performed based on object distance, motion characteristics, and spatial extent. Objects that are close to the ego vehicle and exhibit

strong approaching motion are classified as high threat. Moderately distant objects or those with slower motion are categorized as medium threat, while distant, static, or receding objects are considered low threat.

4.4 Visualization and Bird's Eye View (BEV)

A visualization interface is developed using Matplotlib to generate a Bird's Eye View (BEV) representation of the environment. The ego vehicle is positioned at the origin $(0, 0)$, and radar detections are plotted in a 2D spatial map.

Clustered objects are displayed with bounding boxes, color-coded based on threat level (Red for High, Orange for Medium, Blue for Low). This visualization enables qualitative evaluation of detection accuracy and tracking consistency.

The BEV output serves as a validation tool for the software pipeline and provides a reference for comparing FPGA-based hardware results in subsequent stages.

This chapter detailed the implementation of the radar-only perception pipeline, including parameter configuration, radar data processing, clustering, tracking, and visualization using a Bird's Eye View representation. The software-based implementation serves as a validated reference model for the system, demonstrating the effectiveness of the proposed approach in handling radar data. Building on this foundation, the next chapter focuses on the hardware realization of the pipeline, describing the FPGA-based implementation and architectural optimizations for real-time processing.

Chapter 5

FPGA IMPLEMENTATION

The software-defined radar perception pipeline developed in previous chapters provides a functional validation of noise filtering, clustering, and tracking algorithms. However, software execution introduces latency and lacks deterministic timing, making it unsuitable for real-time embedded applications.

To address these limitations, the radar processing pipeline is implemented on an FPGA using Verilog HDL. The hardware implementation enables parallel processing, reduced latency, and deterministic execution, which are essential for real-time ADAS systems.

The design is implemented on the PYNQ-Z2 platform and follows a modular architecture, where each stage of the radar pipeline is mapped to a dedicated hardware module.

5.1 Overall Hardware Architecture

The overall FPGA architecture of the radar processing pipeline is illustrated in Figure 5.1.The system is composed of multiple interconnected hardware modules forming a complete processing chain.

The architecture follows a sequential data flow:

```
Sweep Memory → Multi-Sweep Accumulator → Range Filter
  → Grid Accumulator → BRAM → Centroid Detection
```

Each module performs a specific function and passes processed data to the next stage.

5.2 Frame-Based Processing Architecture

The implemented FPGA design follows a frame-based processing approach, where radar data is processed in batches across multiple sweeps.

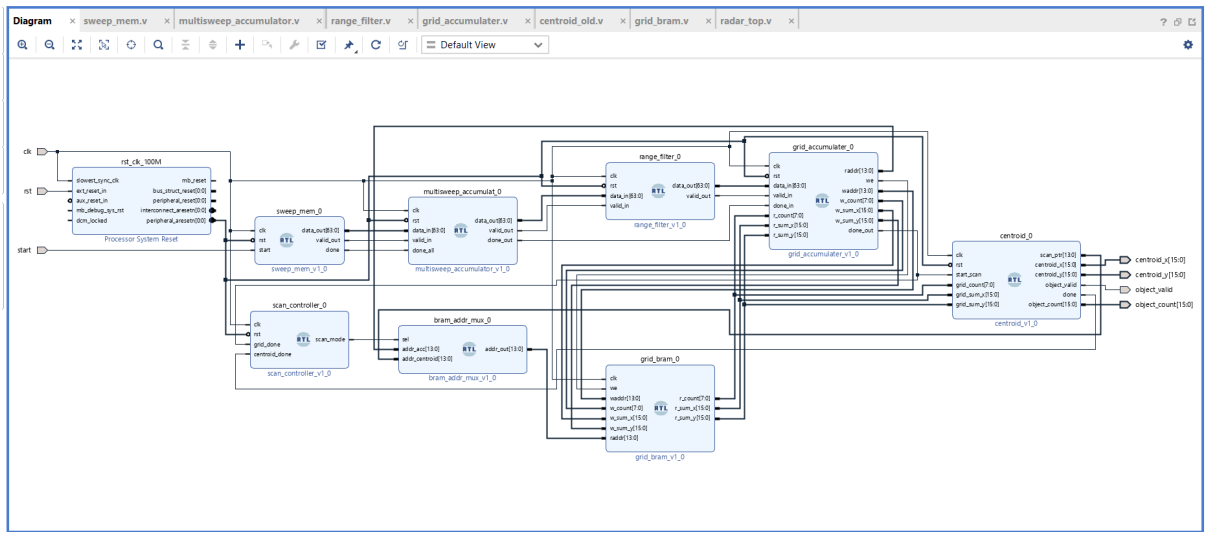

Figure 5.1: FPGA block design of radar processing pipeline

5.2.1 Working Principle

In the frame-based processing approach, radar data is initially loaded from memory files into the FPGA. Multiple radar sweeps are then processed sequentially, and the data from these sweeps is accumulated before further processing. Once the complete frame has been processed, object detection is performed on the accumulated data. This approach closely resembles the software pipeline and serves as a baseline for hardware validation.

5.3 Hardware Module Design

5.3.1 Sweep Memory Module

The sweep memory module is responsible for loading radar data from memory files and streaming it into the processing pipeline. It loads multiple radar sweeps using `$readmemh` and operates using a finite state machine (FSM) with states such as IDLE, LOAD, STREAM, and NEXT. The module streams valid radar points sequentially while generating control signals such as `valid_out` and `done`, thereby acting as the primary data source for the pipeline.

5.3.2 Multi-Sweep Accumulator

The multi-sweep accumulator combines radar points from multiple sweeps to improve point cloud density. Incoming data is stored in internal memory while maintaining appropriate write and read pointers. All sweeps are accumulated before being forwarded, and the accumulated data is output once the `done_all` signal is received. This stage significantly improves detection reliability by increasing data density.

5.3.3 Range Filter Module

The range filter module removes radar points that fall outside the defined perception radius. It extracts the `x` and `y` coordinates from the input data and computes the squared distance as

$$r^2 = x^2 + y^2 \tag{5.1}$$

This value is compared against a predefined threshold corresponding to 60 meters in Q8.8 fixed-point format. Only valid points within the range are passed to the next stage, thereby reducing noise and computational load.

5.3.4 Grid Accumulator Module

The grid accumulator performs spatial clustering by mapping radar points to discrete grid cells. The input coordinates are first converted to grid indices using fixed-point scaling, after which the grid address is computed as

$$addr = (gx + offset) + GRID\_SIZE \times (gy + offset) \tag{5.2}$$

The module reads existing cell data from BRAM and updates key parameters such as point count and the sum of x and y coordinates. This module forms the core of hardware-based clustering.

5.3.5 Grid BRAM

The grid BRAM stores accumulated data corresponding to each grid cell, including point count and the sum of coordinate values. It supports simultaneous read and write operations and is implemented using block RAM to ensure efficient memory utilization.

5.3.6 Centroid Detection Module

The centroid detection module scans all grid cells to identify valid objects. It applies a threshold on the point count and computes the centroid as

$$C_x = \frac{\sum x}{N}, \quad C_y = \frac{\sum y}{N} \tag{5.3}$$

The module generates an `object_valid` signal for detected objects and outputs their positions.

5.3.7 Top-Level Integration

The `radar_top` module integrates all submodules into a unified processing pipeline. It connects the various processing stages, manages control signals between modules, implements scan mode switching between accumulation and centroid detection, and maintains a count of detected objects. This module represents the complete FPGA implementation of the radar perception system.

5.4 Fixed-Point Representation

To enable efficient hardware implementation, fixed-point arithmetic is used instead of floating-point.

5.4.1 Format Used

Q8.8 fixed-point format is used, where the upper 8 bits represent the integer part and the lower 8 bits represent the fractional part. This representation reduces hardware complexity, en-

ables faster computation, and lowers resource utilization compared to floating-point arithmetic.

5.4.2 Advantages

The use of fixed-point representation offers several advantages for hardware implementation. It significantly reduces hardware complexity compared to floating-point arithmetic, enabling simpler and more efficient circuit design. Additionally, it allows faster computation due to reduced arithmetic overhead, which is critical for real-time processing. Furthermore, fixed-point operations lead to lower resource utilization on the FPGA, making the design more efficient and scalable.

5.5 Limitations of Frame-Based Design

Although the frame-based implementation validates the hardware pipeline, it suffers from several limitations, including high latency due to batch processing, inefficient memory usage, and limited real-time capability. These drawbacks motivate the transition to a stream-based processing architecture.

The FPGA implementation successfully maps the radar processing pipeline into hardware using a modular design approach. Each stage of the software pipeline is translated into a dedicated hardware module, enabling efficient processing of radar data.

The frame-based architecture serves as a baseline implementation and validates the correctness of the hardware pipeline, forming the foundation for further optimization using streaming architectures.

5.6 Stream-Based Processing Architecture

To overcome the limitations of frame-based processing, the system is redesigned using a stream-based architecture based on the AXI4-Stream protocol. This approach enables real-time processing by allowing continuous data flow through pipelined hardware modules.

Unlike frame-based processing, where data is stored and processed in batches, the streaming architecture processes radar points as they arrive, significantly reducing latency and improv-

ing throughput.

5.6.1 Overall Streaming Architecture

The streaming-based FPGA architecture is shown in Figure 5.2. The streaming system integrates the FPGA processing pipeline with the Processing System (PS) using AXI DMA. The design is implemented as a custom AXI IP core and connected to the Zynq Processing System.

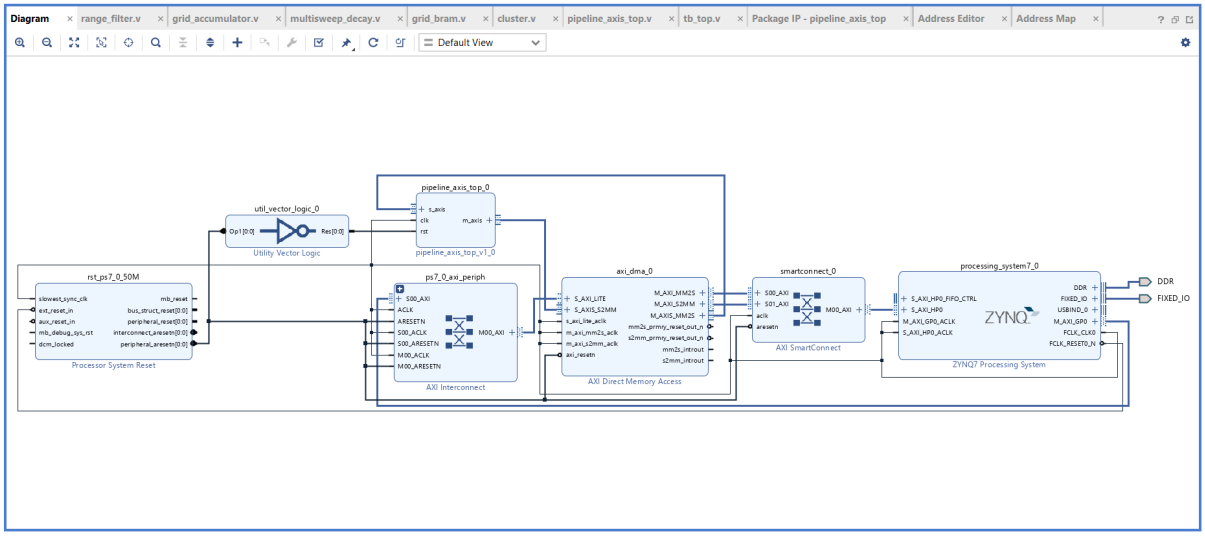

Figure 5.2: AXI streaming-based FPGA architecture with DMA and processing pipeline

The data flow is as follows:

`PS (Python) → AXI DMA → Streaming Pipeline → AXI DMA → PS Output`

5.6.2 AXI4-Stream Interface Design

The pipeline follows the AXI4-Stream protocol for communication between modules. The `tvalid` signal indicates valid data, while `tready` indicates the readiness of the receiving module. The `tdata` signal carries 64-bit radar data, and `tlast` is used to indicate the end of a frame. Data transfer occurs only when both `tvalid` and `tready` are asserted. The TLAST signal plays a critical role in marking frame boundaries and ensuring proper DMA synchronization.

5.6.3 Streaming Pipeline Modules

Range Filter (AXI Version)

The range filter module is redesigned to support AXI streaming and operates using the valid-ready handshake mechanism for continuous data transfer. It computes the squared distance of incoming radar points using 32-bit arithmetic and filters out points that fall beyond the defined 60 m range. Invalid points are discarded by deasserting the `valid_out` signal. Unlike the frame-based implementation, this module processes data continuously without requiring intermediate buffering.

Grid Accumulator (Pipelined)

The grid accumulator is implemented as a multi-stage pipeline to support high-throughput streaming. It converts incoming coordinates into grid indices and computes corresponding memory addresses while handling BRAM latency through staged processing. Write-forwarding logic is incorporated to avoid data hazards and ensure that updated values are immediately available for subsequent operations. The module continuously updates grid cell statistics in real time as data flows through the pipeline.

The pipeline consists of multiple stages, including coordinate decoding, address computation, BRAM read latency handling, intermediate data capture, and final write-back with output generation. This staged architecture enables efficient parallel processing and sustained data throughput.

Multi-Sweep Decay Module

To maintain temporal consistency in the accumulated grid data, a decay mechanism is introduced. The module iterates through all grid cells and applies an exponential decay function given by

$$value = value - \frac{value}{2^k} \tag{5.4}$$

This operation gradually reduces the influence of older detections, ensuring that the system remains responsive to recent radar observations. The decay mechanism effectively replaces explicit multi-frame accumulation used in the frame-based design.

Cluster Detection Module

The cluster detection module scans the grid to identify valid object clusters and generate object-level outputs. It applies a threshold on the grid cell count to detect significant clusters and computes their centroids using reciprocal multiplication, thereby avoiding costly division operations in hardware. The module outputs the detected cluster information using the AXI stream interface and generates the TLAST signal to indicate frame completion. This approach ensures efficient and continuous extraction of object-level information from the streaming pipeline.

5.6.4 Hardware Optimization Techniques

The FPGA implementation incorporates several hardware-specific optimizations to ensure high performance and correctness. A pipelined architecture is used in the grid accumulator to handle BRAM latency and maintain continuous data flow. Write-forwarding logic is implemented to avoid read-after-write hazards by using updated values instead of stale memory data.

Additionally, division operations in centroid computation are replaced with reciprocal multiplication using a lookup table, significantly reducing hardware complexity. The AXI valid-ready handshake mechanism provides effective backpressure handling, preventing data loss under varying processing rates. A pipeline flushing mechanism is also incorporated to ensure that all in-flight data is processed before switching operational modes.

5.6.5 Pipeline Control and Mode Switching

The system operates as a finite state machine with three modes: streaming mode, where radar data is processed in real time; decay mode, where temporal decay is applied to accumulated grid values; and cluster mode, where object-level information is extracted. A pipeline

flush counter ensures that all intermediate stages are cleared before transitioning between modes, thereby maintaining data consistency.

5.6.6 BRAM Access Arbitration

Since multiple modules access BRAM, an address multiplexer is used to manage access between the grid accumulator during write operations, the decay module during read/write operations, and the cluster module during read operations. Control logic ensures conflict-free and efficient memory access.

5.7 Advantages of Streaming Architecture

The stream-based implementation offers several advantages over the frame-based design. It enables real-time processing by allowing continuous data flow through the hardware pipeline, significantly reducing overall system latency. The architecture also improves resource utilization by eliminating the need for large intermediate memory buffers, leading to a more efficient hardware design.

Furthermore, the streaming approach supports continuous data processing, ensuring that incoming radar data can be handled without interruption. This results in higher throughput and better system responsiveness. In addition, the modular and pipelined nature of the streaming architecture enhances scalability, making it suitable for more complex and high-density radar processing scenarios.

5.8 Frame vs Stream Processing Comparison

Feature	Frame-Based	Stream-Based
Processing Style	Batch	Continuous
Latency	High	Low
Memory Usage	High	Low
Throughput	Limited	High
Real-Time Capability	No	Yes

Table 5.1: Comparison between frame and stream processing architectures

The FPGA implementation of the radar-only perception pipeline demonstrates a successful transition from a software-defined model to a hardware-accelerated system capable of real-time operation. Both frame-based and stream-based architectures were developed, with the streaming approach significantly improving latency, throughput, and overall efficiency through continuous data processing and pipelined design. By leveraging AXI4-Stream interfaces, fixed-point optimization, and modular hardware architecture, the system achieves deterministic execution and efficient resource utilization. The complete design is validated on the PYNQ-Z2 platform using AXI DMA, enabling end-to-end processing from software input to hardware-generated output. The next chapter presents the testing methodology and results, evaluating the performance and correctness of both software and FPGA implementations.

Chapter 6

TESTING AND RESULTS

This chapter builds upon the FPGA implementation presented in the previous chapter, where the radar-only perception pipeline was successfully realized in hardware using both frame-based and stream-based architectures. Following the design and implementation stages, it is essential to evaluate the correctness, performance, and real-time capability of the system. This chapter presents a comprehensive analysis of testing procedures and experimental results obtained from both software and hardware implementations.

6.1 Testing and Results

6.1.1 Testing Overview

The testing phase evaluates the correctness and performance of both the software-defined radar processing pipeline and its FPGA-based implementation. The objective is to verify the functionality of filtering, clustering, and object detection across different stages of the system.

The evaluation includes:

- Software-based validation using Python and nuScenes dataset
- Frame-based FPGA simulation
- Stream-based FPGA simulation

Special emphasis is given to the stream-based architecture due to its suitability for real-time processing.

6.1.2 Experimental Setup

The software pipeline was tested using radar point cloud data from the nuScenes-mini dataset in a Python environment. The system processes multi-sweep radar data and performs

clustering, tracking, and threat evaluation.

For hardware validation, both simulation and real hardware execution were performed. FPGA modules were first simulated using Vivado to verify functional correctness. The complete design was then deployed on the PYNQ-Z2 board using AXI DMA for end-to-end validation. The stream-based pipeline was tested using the AXI4-Stream protocol with valid-ready handshake to ensure correct data flow between modules.

Both frame-based and stream-based architectures were evaluated to study differences in latency and processing behavior.

6.1.3 Software-Based Results

The software implementation was validated using both console-based outputs and Bird's Eye View (BEV) visualization. The system successfully performed multi-sweep accumulation, clustering, tracking, and threat classification.

The console output provides detailed information about detected objects, including position, velocity, distance, and threat level for each frame. The corresponding console output is shown in Figure 6.1.

```
[Frame 006] ID=29 | Threat=MEDIUM | Vr= -3.21 m/s | Dist= 39.63 m | BBox(w,h)=(0.20,0.80) | Center=(22.10,32.90) |
[Frame 006] ID=30 | Threat=LOW    | Vr=  0.53 m/s | Dist= 50.39 m | BBox(w,h)=(0.80,0.80) | Center=(39.80,30.90) |
[Frame 006] ID=31 | Threat=MEDIUM | Vr= -1.34 m/s | Dist= 16.95 m | BBox(w,h)=(0.60,0.40) | Center=(9.10,14.30) |
[Frame 006] ID=32 | Threat=LOW    | Vr= -0.00 m/s | Dist= 49.65 m | BBox(w,h)=(0.61,0.20) | Center=(12.23,48.12) |
[Frame 006] ID=24 | Threat=LOW    | Vr=  0.00 m/s | Dist= 22.50 m | BBox(w,h)=(1.27,0.69) | Center=(-19.65,-10.97) |
[Frame 006] ID=12 | Threat=LOW    | Vr=  0.00 m/s | Dist= 54.64 m | BBox(w,h)=(0.07,0.79) | Center=(-53.73,-9.93) |
[Frame 006] ID=19 | Threat=MEDIUM | Vr= -5.36 m/s | Dist= 22.57 m | BBox(w,h)=(1.20,1.60) | Center=(15.40,16.50) |
[Frame 006] ID=14 | Threat=MEDIUM | Vr= -3.57 m/s | Dist= 30.19 m | BBox(w,h)=(0.60,2.00) | Center=(22.70,19.90) |
[Frame 006] ID=20 | Threat=MEDIUM | Vr= -4.97 m/s | Dist= 33.20 m | BBox(w,h)=(1.00,1.20) | Center=(18.90,27.30) |
[Frame 006] ID=21 | Threat=LOW    | Vr=  0.00 m/s | Dist= 57.69 m | BBox(w,h)=(0.40,0.60) | Center=(47.60,32.60) |
[Frame 006] ID=22 | Threat=LOW    | Vr=  0.26 m/s | Dist= 33.47 m | BBox(w,h)=(0.63,2.80) | Center=(15.12,29.86) |
[Frame 006] ID=23 | Threat=LOW    | Vr=  0.00 m/s | Dist= 49.34 m | BBox(w,h)=(0.60,0.19) | Center=(8.22,48.65) |
[Frame 006] ID=17 | Threat=LOW    | Vr=  0.11 m/s | Dist= 47.97 m | BBox(w,h)=(0.42,0.99) | Center=(24.04,41.51) |
[Frame 006] ID=13 | Threat=MEDIUM | Vr= -5.02 m/s | Dist= 24.75 m | BBox(w,h)=(1.00,1.60) | Center=(15.50,19.30) |
[Frame 006] ID=15 | Threat=MEDIUM | Vr= -4.29 m/s | Dist= 35.81 m | BBox(w,h)=(0.00,1.00) | Center=(18.60,30.60) |
[Frame 006] ID=16 | Threat=LOW    | Vr= -0.00 m/s | Dist= 53.71 m | BBox(w,h)=(0.80,0.40) | Center=(36.40,39.50) |
[Frame 006] ID=18 | Threat=LOW    | Vr=  0.00 m/s | Dist= 36.77 m | BBox(w,h)=(0.83,0.33) | Center=(-34.94,-11.44) |
[Frame 007] ID=40 | Threat=HIGH   | Vr= -3.77 m/s | Dist= 15.62 m | BBox(w,h)=(1.00,5.00) | Center=(14.50,5.80) |
[Frame 007] ID=41 | Threat=MEDIUM | Vr= -4.63 m/s | Dist= 23.42 m | BBox(w,h)=(2.20,5.20) | Center=(18.70,14.10) |
[Frame 007] ID=13 | Threat=MEDIUM | Vr= -6.55 m/s | Dist= 25.03 m | BBox(w,h)=(0.20,1.00) | Center=(14.50,20.40) |
[Frame 007] ID=42 | Threat=LOW    | Vr=  0.40 m/s | Dist= 40.70 m | BBox(w,h)=(1.00,1.40) | Center=(36.10,18.80) |
[Frame 007] ID=43 | Threat=LOW    | Vr=  0.23 m/s | Dist= 47.35 m | BBox(w,h)=(1.00,2.00) | Center=(43.50,18.70) |
[Frame 007] ID=44 | Threat=LOW    | Vr= -0.08 m/s | Dist= 54.45 m | BBox(w,h)=(0.40,1.00) | Center=(50.80,19.60) |
[Frame 007] ID=45 | Threat=MEDIUM | Vr= -3.46 m/s | Dist= 35.19 m | BBox(w,h)=(0.00,0.80) | Center=(23.60,26.10) |
[Frame 007] ID=35 | Threat=LOW    | Vr= -0.07 m/s | Dist= 28.20 m | BBox(w,h)=(0.20,0.60) | Center=(24.70,13.60) |
[Frame 007] ID=46 | Threat=LOW    | Vr= -0.19 m/s | Dist= 39.56 m | BBox(w,h)=(0.20,0.80) | Center=(29.90,25.90) |
[Frame 007] ID=03 | Threat=LOW    | Vr=  0.41 m/s | Dist= 28.57 m | BBox(w,h)=(0.79,0.61) | Center=(14.92,24.36) |
```

Figure 6.1: Sample console log showing tracked objects, velocity, distance, and threat level

In addition to console logs, the processed radar data is visualized in Bird's Eye View (BEV) format, providing a top-down spatial representation of the environment. The visualization shows raw radar detections, clustered objects, and tracked targets along with velocity-based classification, enabling clear interpretation of object positions and motion characteristics.

The BEV output for a sample frame is shown in Figure 6.2.

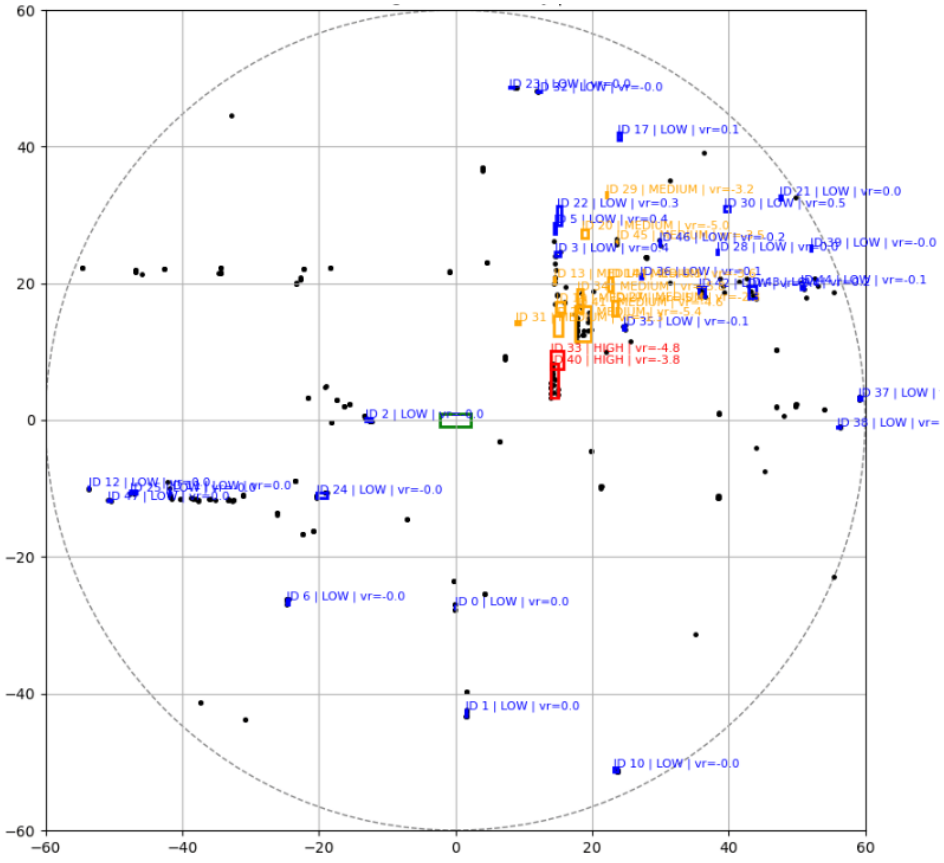

Figure 6.2: Bird's Eye View (BEV) representation of radar point cloud showing clustered objects with tracking IDs and radial velocity classification

6.1.4 Frame-Based FPGA Simulation Results

The frame-based FPGA architecture was simulated using Vivado to validate functional correctness. In this approach, radar data is processed in batches across multiple sweeps before clustering is performed.

The simulation results confirm that radar points are correctly buffered and accumulated

over a complete frame, and grid-based accumulation accurately stores hit counts and coordinate sums. Clustering is performed after full frame acquisition, and detected objects are successfully output with centroid and count information. The console output further validates correct operation by showing successful execution of each stage in a sequential, frame-based manner.

The console output further validates correct operation, showing successful execution of each stage:

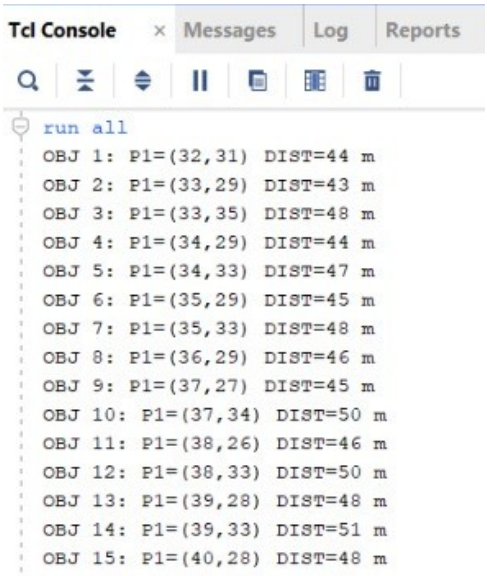

Figure 6.3: Frame-Based FPGA Simulation Console Logs

These results indicate that each module in the pipeline operates correctly in a sequential, frame-based manner.

However, due to batch processing, this approach introduces higher latency, making it less suitable for real-time applications.

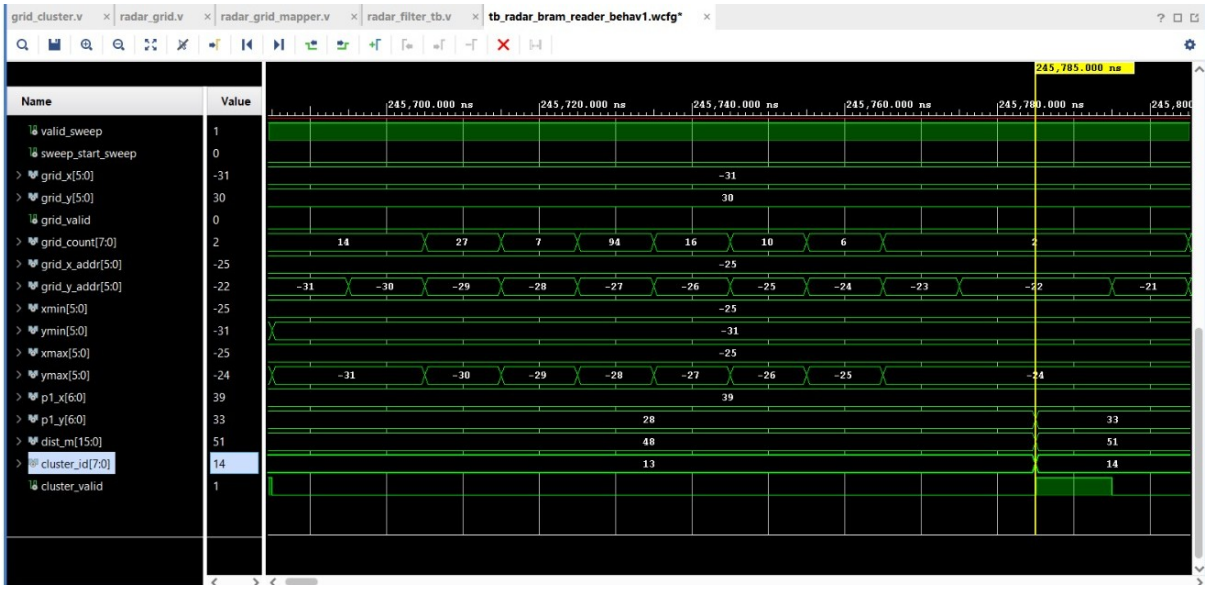

Figure 6.4: Frame-Based FPGA Simulation Output with Console Logs

6.1.5 Stream-Based FPGA Simulation Results

The stream-based FPGA architecture was implemented using the AXI4-Stream protocol and simulated in Vivado. Unlike the frame-based approach, this design processes data continuously without requiring full-frame buffering.

The simulation verifies correct AXI handshake operation using `tvalid` and `tready`, along with proper propagation of the `tlast` signal for frame boundary detection. The pipeline supports continuous streaming of radar data through all processing stages, enabling real-time grid accumulation and clustering without stalling.

The console output provides detailed insight into detected clusters:

```
TIME=1560 | CLUSTER #1 DETECTED
AVG_X = 2560  AVG_Y = 3849  COUNT = 2

TIME=4250 | CLUSTER #2 DETECTED
AVG_X = 5120  AVG_Y = 6719  COUNT = 3
```

The console output confirms that cluster centroids are correctly computed in Q8.8 format and that multiple clusters can be detected within a single data stream. It also demonstrates

effective noise filtering and accurate object detection. Waveform analysis further verifies stable pipeline operation, correct synchronization between input and output streams, and proper timing of cluster outputs after processing latency.

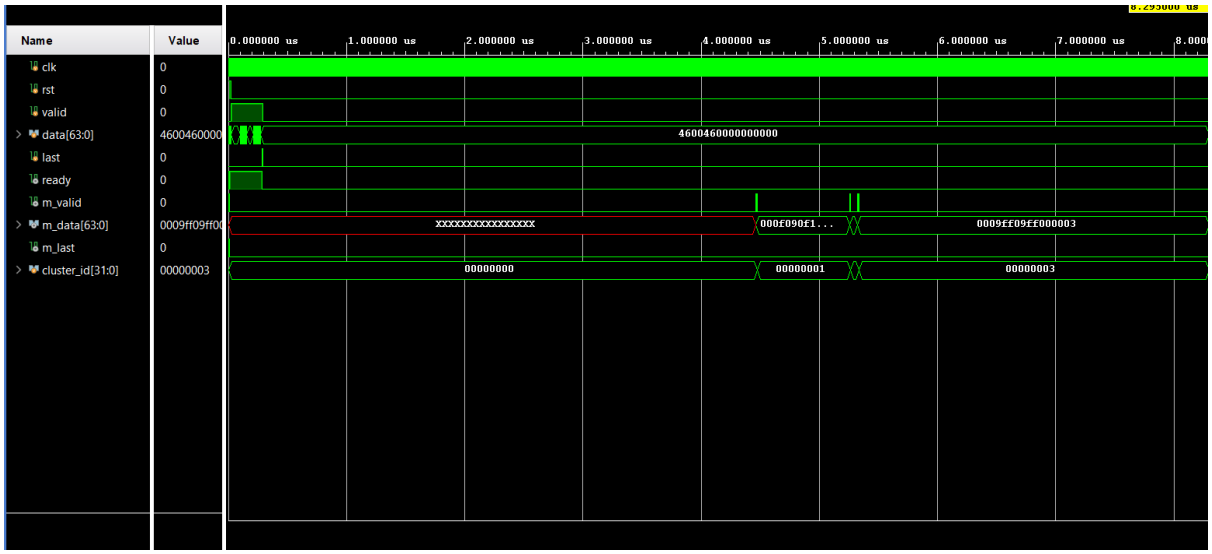

Figure 6.5: Streaming FPGA Pipeline Simulation with Cluster Output

6.1.6 PYNQ-Based Hardware Execution Results

To validate the real-time performance of the FPGA-based radar processing pipeline, the design was deployed on the PYNQ-Z2 board using AXI DMA-based communication between the Processing System (PS) and Programmable Logic (PL).

Synthetic radar point cloud data consisting of multiple well-separated clusters with added noise was generated in Python and transmitted to the FPGA using the PYNQ framework. The input data was converted into fixed-point (Q8.8) representation and packed into 64-bit format before streaming via AXI DMA.

The FPGA processed the incoming data stream and returned clustered outputs through the DMA receive channel. The received data was decoded back into floating-point format on the PS side.

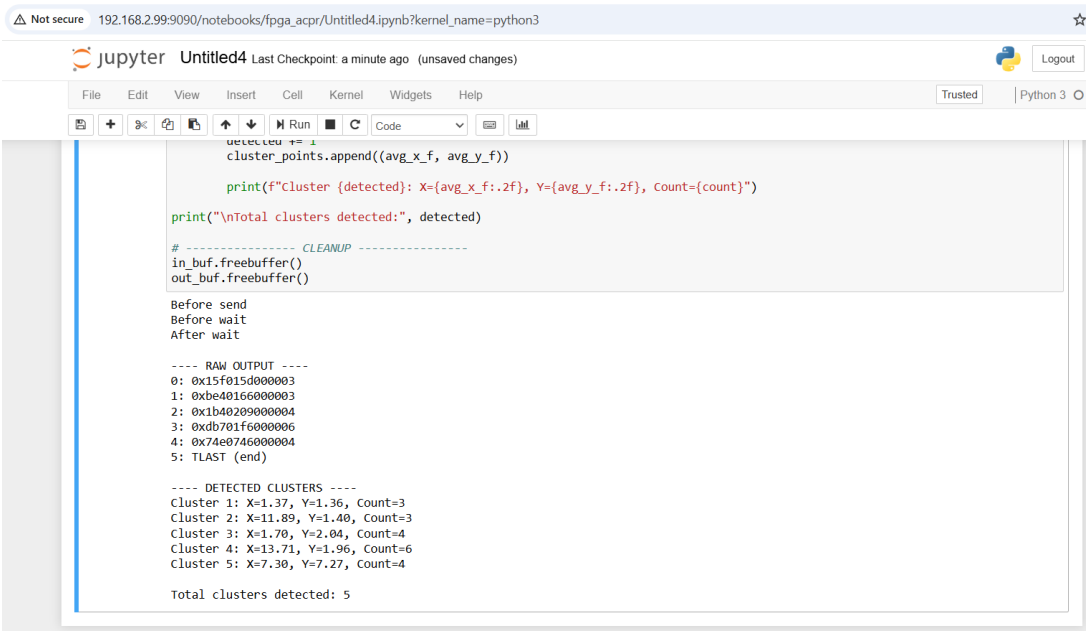

Figure 6.6: PYNQ execution showing raw DMA output and detected cluster results

The console output demonstrates successful end-to-end execution of the pipeline. The FPGA correctly identifies cluster centroids and the number of points associated with each cluster.

The console output demonstrates successful end-to-end execution of the pipeline. The FPGA correctly identifies cluster centroids and the number of points associated with each cluster. The results confirm successful AXI DMA communication between the Processing System and Programmable Logic, along with correct fixed-point to floating-point conversion.

A total of five clusters are detected, matching the input configuration, and their centroids are accurately estimated. The system also correctly handles the TLAST signal to indicate the end of each frame. The detected cluster centers closely match the original input cluster locations, confirming the correctness of the FPGA implementation under real hardware conditions.

This experiment validates the complete processing chain:

Python (PS) → AXI DMA → FPGA (PL) → AXI DMA → Python Output

The results demonstrate that the proposed streaming architecture is successfully implemented on hardware and is capable of real-time radar data processing.

6.1.7 Performance, Power, and Area (PPA) Analysis

The FPGA implementation was evaluated using Vivado implementation reports to analyze resource utilization, power consumption, and timing performance.

Resource Utilization (Area)

The resource utilization report provides insight into the hardware area consumed by the design.

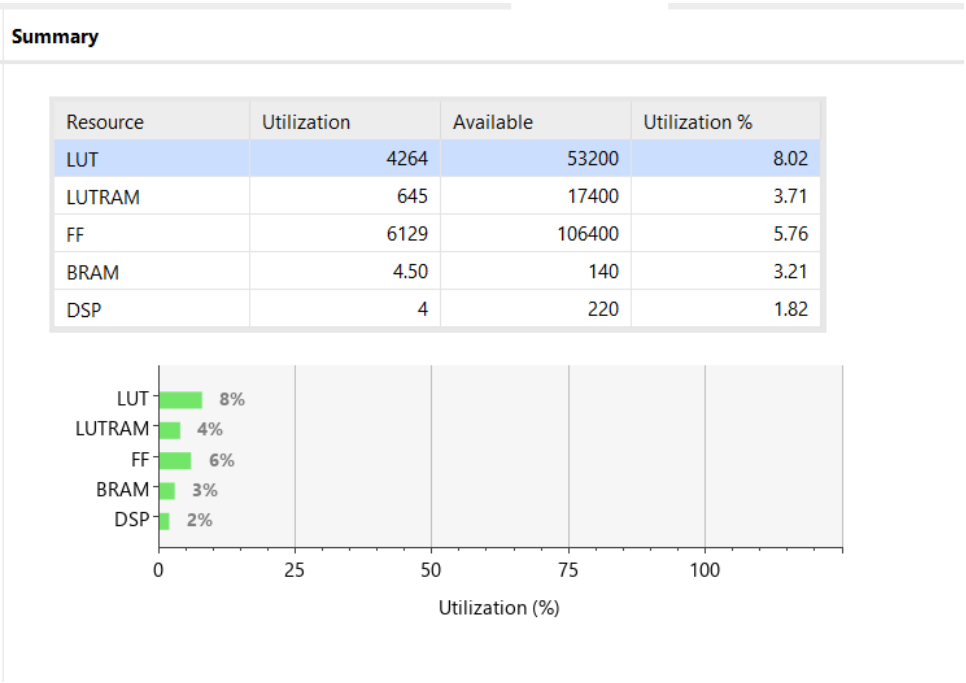

Figure 6.7: FPGA Resource Utilization Report

The design utilizes a small percentage of available FPGA resources, indicating an efficient hardware implementation. LUT utilization is approximately 8%, reflecting moderate combinational logic usage, while Flip-Flop (FF) utilization is around 5–6%, corresponding to pipeline register usage.

BRAM usage is minimal, demonstrating efficient memory utilization, and DSP utilization is very low due to the use of fixed-point arithmetic instead of complex operations. These results confirm that the design is lightweight and scalable for larger implementations.

Power Analysis

The power report estimates the on-chip power consumption of the FPGA design.

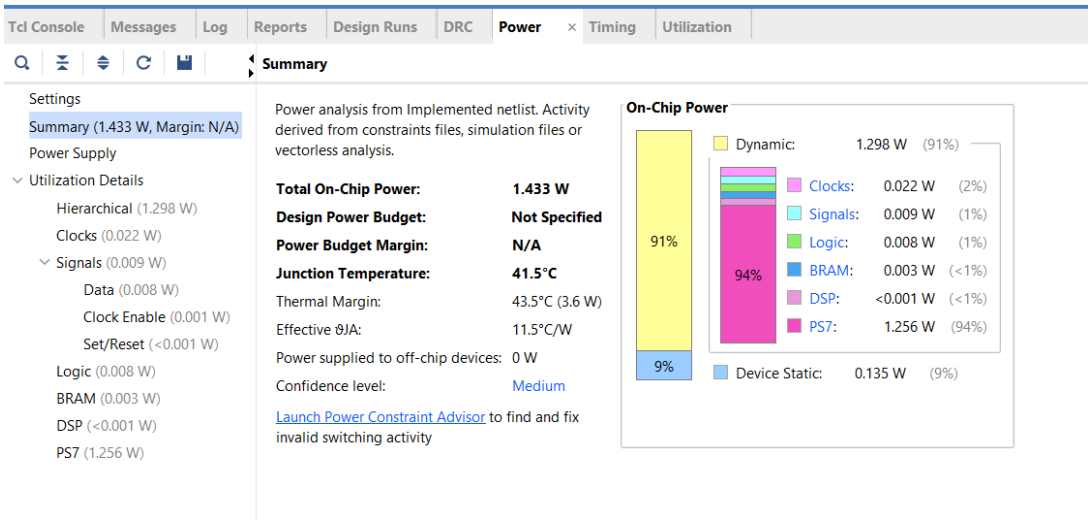

Figure 6.8: FPGA Power Consumption Report

The total on-chip power consumption is approximately 1.43 W. Dynamic power constitutes the major portion of the overall power consumption, while the Processing System contributes significantly to total usage. In comparison, logic, BRAM, and DSP components consume relatively low power. These observations indicate that the hardware accelerator is power-efficient and well suited for embedded applications.

Timing Analysis (Performance)

The timing report evaluates whether the design meets timing constraints for reliable operation.

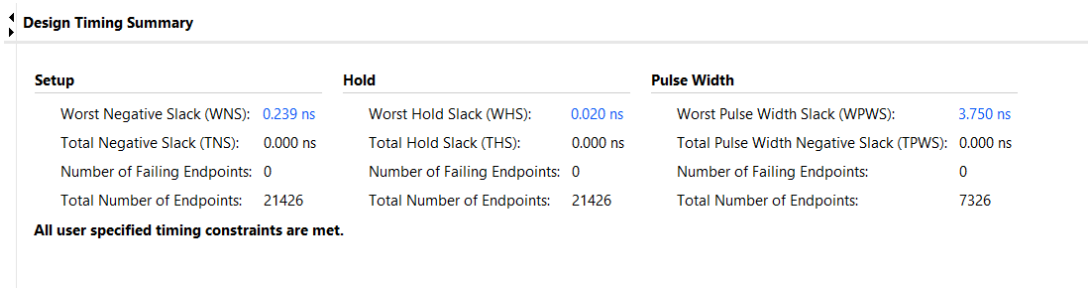

Figure 6.9: FPGA Timing Summary Report

The timing analysis confirms that the design meets all timing constraints required for reliable operation. The Worst Negative Slack (WNS) is positive at 0.239 ns, indicating the absence of timing violations, while the Total Negative Slack (TNS) is zero, confirming complete timing closure. These results demonstrate that the design can operate reliably at the target clock frequency.

Overall PPA Observation

The PPA results demonstrate that the FPGA implementation achieves efficient resource utilization with low area consumption, low power consumption suitable for embedded systems, and successful timing closure ensuring high performance. These outcomes validate the effectiveness of the streaming architecture and confirm its suitability for real-time radar processing applications.

6.1.8 Performance Discussion

The performance of the system was analyzed by comparing frame-based and stream-based architectures. The frame-based approach introduces latency due to batch processing, whereas the stream-based architecture enables continuous data flow. The use of pipelined processing significantly improves throughput, making the streaming architecture more suitable for real-time radar processing applications.

6.1.9 Overall Observation

The experimental results confirm the correctness of the radar processing pipeline across software implementation, FPGA simulation, and real hardware execution. The stream-based FPGA architecture demonstrates superior performance compared to the frame-based approach, particularly in terms of reduced latency, improved throughput, and enhanced real-time processing capability.

Furthermore, the successful integration of the system with the PYNQ-Z2 platform using AXI DMA communication establishes the practical feasibility of deploying radar perception pipelines on FPGA-based embedded systems.

This chapter evaluated the performance of the radar-only perception pipeline across software validation, FPGA simulation, and real hardware execution. The results confirm the correctness of filtering, clustering, and object detection stages, while also highlighting the advantages of the stream-based architecture in achieving low latency and high throughput. The PPA analysis further demonstrates efficient resource utilization, low power consumption, and successful timing closure. These findings validate the effectiveness of the proposed system and its suitability for real-time applications. The next chapter concludes the work by summarizing the overall contributions, discussing limitations, and outlining future enhancements.

Chapter 7

CONCLUSION

This chapter builds upon the comprehensive design, implementation, and evaluation presented in the previous chapters, where the radar-only perception pipeline was developed and validated across both software and FPGA-based hardware platforms. Having demonstrated the effectiveness and real-time capability of the proposed system, this chapter summarizes the key contributions of the work, discusses its limitations, and outlines potential directions for future improvements and extensions.

7.1 Conclusion

This project successfully demonstrates the design and implementation of a radar-only perception pipeline for autonomous systems, integrating both software and hardware domains into a unified framework. The software implementation validates the effectiveness of multi-sweep accumulation, noise filtering, density-based clustering, tracking, and threat assessment using real-world radar data from the nuScenes dataset, producing a structured representation of the environment. The processing pipeline is further translated into an FPGA-based implementation using Verilog HDL, enabling parallel processing and deterministic execution. Both frame-based and stream-based architectures are developed and evaluated, with the stream-based design, implemented using AXI4-Stream protocol and pipelined modules, achieving improved latency and continuous data processing suitable for real-time applications. The complete system is successfully deployed on the PYNQ-Z2 platform using AXI DMA communication between the Processing System and Programmable Logic, demonstrating correct end-to-end functionality including data transfer, hardware execution, and cluster detection. FPGA performance analysis confirms efficient resource utilization, low power consumption, and successful timing closure, while hardware results closely match software behavior, validating the overall design.

This work establishes the feasibility of real-time radar perception using FPGA-based systems and highlights the effectiveness of hardware-accelerated processing for autonomous applications.

7.2 Limitations

Although the FPGA design is successfully validated on the PYNQ-Z2 board, testing is currently limited to synthetic input data rather than full-scale real-world radar datasets such as nuScenes. This restricts the evaluation of the system under diverse and complex real-world driving scenarios.

The current hardware implementation primarily focuses on clustering and centroid detection, and does not include advanced modules such as object tracking, velocity estimation, and threat classification on FPGA. In addition, the clustering approach is grid-based and threshold-driven, which may be less adaptive compared to advanced machine learning or adaptive density-based methods.

Furthermore, resource utilization and scalability have not been evaluated for larger grid sizes or high-density radar scenarios. End-to-end latency and throughput measurements on hardware have also not been quantitatively benchmarked, limiting a complete performance analysis.

Finally, the system processes data frame-wise per DMA transaction, and continuous multi-frame real-time streaming can be further optimized to improve overall system efficiency and responsiveness.

7.3 Future Scope

Future enhancements to this work can focus on extending both the algorithmic capabilities and hardware performance of the radar perception system. One key direction is the integration of real-world radar datasets such as nuScenes directly with FPGA hardware to enable complete end-to-end validation under practical operating conditions.

Further improvements include extending the hardware implementation to incorporate ad-

vanced modules such as object tracking, velocity estimation, and threat assessment, enabling a fully hardware-accelerated perception pipeline. Optimization of AXI streaming and DMA architecture can also support continuous multi-frame real-time processing with reduced latency.

In addition, incorporating advanced algorithms such as improved DBSCAN variants, Kalman Filters, or deep learning-based approaches can enhance detection accuracy and tracking robustness. Hardware-level optimizations using High-Level Synthesis (HLS), AI accelerators, and parallel processing techniques can further improve performance, scalability, and resource efficiency.

Finally, the system can be extended to support sensor fusion by integrating data from cameras and LiDAR, enabling robust multi-modal perception for autonomous systems.

7.4 Future Scope

Overall, this work demonstrates a complete and practical approach to radar-only perception, bridging the gap between algorithmic development and hardware realization. By combining robust signal processing techniques with efficient FPGA-based acceleration, the project establishes a strong foundation for real-time autonomous perception systems. The insights and outcomes of this work provide a basis for further research and development in scalable, high-performance radar processing architectures for next-generation intelligent systems.

Bibliography

[1] R. Schmidt, J. Wenger, and T. Zwick, "Automotive radar systems for advanced driver assistance systems," *IEEE Microw. Mag.*, vol. 18, no. 1, pp. 24–40, Jan. 2017.

[2] D. Hunt, M. McCracken, J. Chen, and R. W. Heath, "RadCloud: Real-time high-resolution point cloud generation using low-cost radars for aerial and ground vehicles," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Yokohama, Japan, May 2024, pp. 1–8.

[3] S. Savazzi, V. Rampa, S. Kianoush, A. Minora, and L. Costa, "Cooperation and federation in distributed radar point cloud processing," arXiv preprint arXiv:2405.01995, May 2024. [Online]. Available: https://arxiv.org/abs/2405.01995

[4] K. Luan, C. Shi, N. Wang, Y. Cheng, H. Lu, and X. Chen, "Diffusion-based point cloud super-resolution for mmWave radar data," arXiv preprint arXiv:2404.06012, Apr. 2024. [Online]. Available: https://arxiv.org/abs/2404.06012

[5] M. Cen and P. Newman, "Precise ego-motion estimation with millimeter-wave radar under diverse and challenging conditions," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Brisbane, Australia, May 2018, pp. 1–8.

[6] S. S. Blackman and R. Popoli, *Design and Analysis of Modern Tracking Systems*. Norwood, MA, USA: Artech House, 1999.

[7] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

[8] A. H. Sayed, *Adaptive Filters*. Hoboken, NJ, USA: Wiley, 2008.

[9] A. Danzer, T. Griebel, M. Bach, and K. Dietmayer, "Radar-based object detection and tracking for autonomous driving," in *Proc. IEEE Intell. Veh. Symp. (IV)*, Paris, France, Jun. 2019, pp. 1–8.

[10] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 652–660.

[11] M. Ester, H. P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. ACM Int. Conf. Knowl. Discovery Data Mining (KDD)*, Portland, OR, USA, Aug. 1996, pp. 226–231.

[12] M. Braun, A. Cennamo, M. Schoeler, K. Kollek, and A. Kummert, "Semantic segmentation of radar detections using convolutions on point clouds," arXiv preprint arXiv:2305.12775, May 2023. [Online]. Available: https://arxiv.org/abs/2305.12775

[13] J. Schumann, M. Hahn, and C. Stiller, "Scene understanding based on clustering of dynamic objects using automotive radar," in *Proc. IEEE Intell. Transp. Syst. Conf. (ITSC)*, Maui, HI, USA, Nov. 2018, pp. 1–8.

[14] Xilinx, Inc., "AXI DMA v7.1 LogiCORE IP Product Guide," San Jose, CA, USA, Doc. PG021, 2021. [Online]. Available: `https://docs.xilinx.com/r/en-US/pg021_axi_dma`

[15] Xilinx, Inc., "AXI4-Stream Protocol Specification," San Jose, CA, USA, Doc. UG761, 2020. [Online]. Available: `https://docs.xilinx.com/v/u/en-US/ug761_axi_reference_guide`

[16] W. Wolf, "FPGA-based system design for real-time embedded signal processing," *IEEE Des. Test Comput.*, vol. 25, no. 5, pp. 426–433, Sep.–Oct. 2008.

[17] D. González, J. Pérez, V. Milanés, and F. Nashashibi, "A review of motion planning techniques for automated vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 4, pp. 1135–1145, Apr. 2016.

[18] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A multimodal dataset for autonomous driving," in

Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Seattle, WA, USA, Jun. 2020, pp. 11621–11631.

[19] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Providence, RI, USA, Jun. 2012, pp. 3354–3361.

Appendix A

Hardware and Software Specifications

A.1 PYNQ-Z2 FPGA Board

The PYNQ-Z2 board is a Zynq-7000 based FPGA development platform that integrates a dual-core ARM Cortex-A9 processing system (PS) with programmable logic (PL) on a single chip. This heterogeneous architecture enables efficient hardware-software co-design, allowing computational tasks to be partitioned between software execution on the processor and hardware acceleration on the FPGA fabric. Such a design is highly suitable for real-time embedded signal processing applications, including radar-based perception systems.

The board is built around the Xilinx XC7Z020 SoC and provides support for AXI4 and AXI4-Stream interfaces, which are used in this project for high-speed data transfer between software and hardware modules. Direct Memory Access (DMA) is employed to facilitate efficient streaming of radar data between the Processing System and Programmable Logic, minimizing processor overhead and enabling continuous data flow.

Key Specifications:

- **FPGA Device:** Xilinx Zynq-7000 SoC (XC7Z020-1CLG400C)
- **Processing System:** Dual-core ARM Cortex-A9 processor (up to 650 MHz)
- **Programmable Logic:** 85K logic cells, 220 DSP slices, 560 KB block RAM
- **Memory:** 512 MB DDR3 RAM shared between PS and PL
- **Non-Volatile Storage:** microSD card support for boot and data storage
- **Interfaces:** AXI4, AXI4-Lite, and AXI4-Stream for high-speed communication
- **DMA Support:** AXI DMA for efficient data transfer between PS and PL
- **Clocking:** Programmable clock generators for flexible timing control
- **Connectivity:** Gigabit Ethernet, USB 2.0, HDMI input/output

- **Peripheral Interfaces:** GPIO, I2C, SPI, UART, PWM
- **Expansion:** Arduino headers and PMOD connectors for external modules
- **Power Supply:** 5V DC input with onboard regulation

Reference Documents:

- https://reference.digilentinc.com/reference/programmable-logic/pynq-z2/start
- https://pynq.readthedocs.io/en/latest/

A.2 nuScenes Dataset

The nuScenes dataset is a large-scale autonomous driving dataset designed for research in perception, sensor fusion, and object tracking. It provides synchronized data from multiple sensors including cameras, LiDAR, and radar.

In this project, radar data from five sensors is utilized to achieve near 360-degree environmental perception around the ego vehicle.

Radar Sensors Used:

- RADAR_FRONT
- RADAR_FRONT_LEFT
- RADAR_FRONT_RIGHT
- RADAR_BACK_LEFT
- RADAR_BACK_RIGHT

The dataset provides spatial and motion-related information required for object detection and tracking.

Key Features:

- Multi-sensor data (Camera, LiDAR, Radar)
- 360-degree environmental coverage

- Time-synchronized sensor data
- Real-world urban driving scenarios

Reference Documents:

- https://www.nuscenes.org/
- https://github.com/nutonomy/nuscenes-devkit

A.3 Radar Sensor Data and Fields

Radar data in the nuScenes dataset contains spatial and velocity information required for radar-only perception. Each radar point includes ego-compensated Doppler velocity for motion estimation.

Important Radar Data Fields:

Field	Description
x, y, z	Position coordinates
vx, vy	Velocity components (m/s)
rcs	Radar Cross Section
dyn_prop	Dynamic property classification
id	Object identifier
timestamp	Time of measurement

Table A.1: Radar data fields used in the perception pipeline

In both software and hardware implementations, spatial coordinates are extracted for clustering and object detection.

In the software implementation, the position coordinates are obtained as:

```
x = pts[0, :]
y = pts[1, :]
```

In the hardware implementation, these values are encoded in the radar data stream and processed using fixed-point (Q8.8) representation. The x and y coordinates are used for range filtering, grid accumulation, and centroid computation.These coordinates form the basis for object localization and are essential for clustering and tracking in the radar perception pipeline.

Reference Documents:

- https://www.nuscenes.org/data-collection
- https://github.com/nutonomy/nuscenes-devkit